Prompt-Based Collaborative Agent Framework (PCAF) for Enhanced LLM Mathematical Reasoning

Petros Tsitsekidis
Department of Computer Science
Aarhus University
au802942@uni.au.dk

Abstract

The inherent unreliability of Large Language Models (LLMs) in complex, multi step calculations necessitates architectural interventions to achieve trustworthy accuracy in mathematical reasoning. This paper introduces the Prompt Based Collaborative Agent Framework (PCAF), a novel, architecture driven approach aimed at significantly improving final answer accuracy on the MathArena benchmark suite, including AIME 2025, HMMT (Feb/Nov) 2025, BRUMO 2025, SMT 2025, and CMIMC 2025. The method utilizes a single LLM instance (DeepSeek-V3-03-24) and employs sequential, role specific prompting to simulate a sophisticated, three agent collaborative system (Solver, Verifier, and Planner). This architecture transforms the LLM's linear reasoning into a persistent self correction mechanism. Through systematic evaluation across six distinct competition formats, we establish that the PCAF's structured intervention yields a significant uplift in global accuracy compared to the MathArena Zero Shot CoT baseline (53.65% versus 39.5%), with a 36.1% recovery rate confirming the efficacy of deterministic, architecture driven reflection.

1 Introduction

The pursuit of reliable mathematical reasoning in Large Language Models is hindered by two persistent challenges: computational imprecision and stubbornness in self correction. While LLMs excel at generating coherent solution narratives (Chain of Thought), they frequently fail at the final, precise steps during arithmetic, algebra, and complex counting, which leads to high rates of final answer error on benchmarks like MathArena. Furthermore, models typically struggle to effectively diagnose and pivot from their initial logical errors, making standard self correction techniques unreliable.

To overcome these limitations, this project implements the Prompt Based Collaborative Agent Framework (PCAF). Instead of relying on model fine tuning, PCAF utilizes an architecture driven reasoning approach. It simulates a multi agent system within a single LLM instance through sequential, role specific prompting. The framework dynamically cycles the LLM through three specialized roles. The Solver, the Verifier and the Planner which work in an iterative, closed loop refinement process. The core objective is to demonstrate that this structured collaborative architecture significantly increases the final answer accuracy and trustworthiness compared to the model's standard Zero Shot baseline provided by MathArena.

The key contributions of this work are:

- **Novel Algorithmic Structure:** We introduce the PCAF loop, which formalizes the iterative roles of Generation (Solver), Critique (Verifier), and Coordination (Planner), forcing systematic logical scrutiny.

Preprint.

- **Enforced Grounding:** We implement the mandatory integration of a persistent Python sandbox environment ("Code" tool) to enforce dual method verification (analytical vs. naive brute force), ensuring mathematical conclusions are derived from verifiable, executed code.
- **Targeted Improvement:** We apply and evaluate PCAF on a broad spectrum of challenging final answer subsections of the MathArena benchmark to provide a clear, quantitative measure of accuracy uplift across varied mathematical domains.

2 Related Work

The Prompt-Based Collaborative Agent Framework (PCAF) is informed by two established and growing areas of research in LLMs: Reflective Reasoning and Verification and Multi-Agent Systems for Problem Solving.

2.1 Reflective Reasoning and Verification

Early methods in mathematical reasoning focused on generating a single, detailed Chain-of-Thought (CoT). However, recent work emphasizes enhancing reliability through self-critique. Approaches such as those presented by Wang et al. (2025) [3], who introduce a collaboration mechanism for long CoT reasoning, and Li et al. (2025) [1], who explores LLM agents for automated proof generation using knowledge graphs, highlight the value of structured introspection. These systems often succeed in generating multiple solution paths or utilizing formal methods to check internal consistency. However, a common limitation is the nature of the corrective feedback; the system may identify an error but lack a dedicated, high-level mechanism to effectively re-plan the solution strategy. The feedback loop often remains constrained to simple re-prompts or voting systems.

2.2 Multi-Agent Systems and Formal Verification

A parallel line of inquiry involves distributing tasks among distinct, specialized LLM instances to simulate collaborative environments. Works like Varambally et al. (2025) [2], which describe a system for recursively building formal proofs, employ specialized agents (a "formal prover" and a "critic") to ensure soundness. This multi-model approach is computationally powerful and minimizes agent bias but requires the coordination and hosting of multiple distinct models. Furthermore, when these systems interact with external tools, such as the formal prover used , the communication requires complex integration layers.

3 Methodology

3.1 Framework Overview

The PCAF operates as a deterministic, closed-loop system designed to enforce self-correction. The architecture operates with a fixed budget of $T = 4$ total turns (one initial generation followed by up to three correction cycles).

The core cycle involves three sequential steps:

1. **Attempt (Solver):** The Solver generates a solution and a corresponding Python execution trace.
2. **Verification (Verifier):** The Verifier audits the trace against a structured checklist.
3. **Correction (Planner):** If the Verifier rejects the solution, the Planner translates the critique into a mandatory, actionable instruction for the next attempt.

The system employs an **Early Stopping** mechanism: the loop terminates immediately if the Verifier outputs a valid signal, minimizing token usage.

3.2 The Agents

3.2.1 The Solver

The Solver is the generative component, constrained by a strict system prompt `get_solver_prompt` that enforces the Dual-Verification Protocol for grounding and reliability.

"Two Strictly Different Methods" Mandate: The Solver must use two conceptually distinct approaches to arrive at the same answer:

- **Analytical Method (Method 1):** Utilizes formal mathematics, algebra, or efficient libraries (e.g., `sympy`).
- **Naive Brute-Force (Method 2):** Requires a simple, unoptimized Python script to simulate or iterate through all possibilities.

"Code-First" Mandate: The Solver is required to execute the code for both methods and explicitly compare the outputs, ensuring the final answer is a value confirmed by an external tool. This process is crucial for preventing numerical hallucinations.

3.2.2 The Verifier

The Verifier's role is that of a Skeptical Math Auditor (`VERIFIER_ROLE`). It receives the Solver's full reasoning and code outputs and audits the solution based on a strict checklist. To ensure the output is machine-parsable and deterministic, the Verifier is constrained via the LLM API's **JSON mode** (`response_format={"type": "json_object"}`) to output a strict schema:

```
{
  "valid": boolean,
  "error_type": "LOGIC_FLAW" | "VALUE_MISMATCH" | "METHOD_CONFLICT",
  "critique": string
}
```

The `error_type` field serves as the primary signal for the Planner, classifying failures into three key categories:

`LOGIC_FLAW` A fundamental error in the mathematical approach or application of a principle (e.g., misapplying the Principle of Inclusion-Exclusion or failing to handle edge cases like division by zero).

`METHOD_CONFLICT` A direct contradiction between the two mandated methods. This flag is triggered if the Analytical derivation yields one result (e.g., 42) while the Naive Brute-Force simulation yields another (e.g., 40).

`VALUE_MISMATCH` The Python code executed correctly, but the Solver's final textual conclusion ignored the code's output. This detects instances where the model "hallucinates" a preferred number despite contradictory empirical evidence from the sandbox.

3.2.3 The Planner

The Planner is a deterministic function (`construct_planner_feedback`) that routes error signals into rigid instructional templates. Its primary goal is to break the "reasoning inertia" often observed in LLMs. The logic implements three routing strategies:

`Syntax Repair (for Runtime Errors)` If the execution trace contains runtime errors (captured via the sandbox's standard error buffer, e.g., `IndentationError` or `Timeout`) or fails to produce output, the Planner ignores the mathematical content entirely. Instead, it issues a strict syntax directive: "Fix the syntax. Make the code run."

`Protocol Reset (for VALUE_MISMATCH)` To counteract the model's tendency to cling to a hallucinated number despite contradictory code output, this flag triggers a "Memory Wipe." The Planner instructs the Solver to: "Ignore your previous text intuition. It was wrong. Re-execute code and accept the output as truth."

`General Critique Relay (for LOGIC_FLAW / METHOD_CONFLICT)` For fundamental reasoning errors or conflicts between the two methods, the Planner avoids enforcing rigid strategy shifts (e.g., forcing Coordinate Geometry), as experimentation showed this often led to the model attempting to "game" the system (hardcoding). Instead, it relays the Verifier's specific critique and mandates a resolution of the inconsistency between the text and the code.

3.3 Tool Use

The execution of mathematical code is handled by a custom, stateful environment, the `PersistentSolverSandbox`. This tool is essential for providing verifiable grounding:

Persistent Environment: Unlike typical stateless code execution tools, the sandbox maintains its global state (`self.globals`) across consecutive code blocks within a single Solver attempt. This enables complex, multi-block workflows in which a function is defined in one block and then invoked or modified in subsequent blocks.

Comprehensive Imports: The sandbox is pre-loaded with essential mathematical and symbolic libraries (`math`, `sympy`, `numpy`) to support both analytical and naive solution methods.

Robust I/O Capture: The sandbox reliably captures all standard output (`stdout`) and handles common runtime issues (e.g., infinite loops via strict 20-second timeouts, `IndentationError`, and `SyntaxError`), returning any error messages to the Solver as an `OBSERVATION`. This feedback loop is critical, as it allows the Solver to debug its own code syntax and execution failures.

4 Experiments

The central goal of these experiments is to quantitatively assess the performance gain of the PCAF architecture relative to the standard MathArena baseline [4]. By applying PCAF to subsets where the baseline model either has established scores (AIME, HMMT Feb) or there is no score available (BRUMO, SMT, CMIMC, HMMT Nov), we establish a rigorous comparative framework to measure the impact of multiagent collaboration on final answer accuracy.

4.1 Experimental Setup

Implementation. The framework utilizes the `DeepSeek-V3-0324` model hosted on Azure OpenAI. The architecture consists of three specialized agents. The Solver, the Verifier and the Planner, all operating within a single LLM context. A key component is the Solver's `PersistentSolverSandbox`, which maintains variable state across turns to allow for iterative code refinement.

Evaluation Protocol. To account for the stochastic nature of LLMs, we adopted a robust evaluation strategy. For each problem in the test set, we performed $K = 4$ independent runs of the full PCAF loop. The reported accuracy for a problem is the average success rate across these runs ($Score \in [0.0, 1.0]$).

Metrics. We assessed performance using three metrics:

- **Accuracy (Pass@K):** The average correctness of the final integer answer across the 4 independent runs.
- **Resilience (Recovery Rate):** The percentage of correct runs where the system initially failed (Turn 1) but successfully self corrected in subsequent turns (Turn 2–4).
- **Baseline Comparison:** Performance is compared against the MathArena's Zero Shot CoT baseline using the same underlying model.

5 Results

As shown in Table 1, the PCAF architecture consistently outperforms the available MathArena baselines. On the HMMT February subset, the model's accuracy improved from 29.0% to 45.83%, representing a 16.8% absolute uplift. Globally, the PCAF framework raised the model's average accuracy from 39.5% to 53.65%.

5.1 Quantitative Performance

Table 1 summarizes the performance metrics on the AIME 2025 and HMMT February 2025 datasets. We report the **Pass@4** accuracy (average score across 4 independent runs per problem) to account for generation variance.

Table 1: Performance of PCAF vs. MathArena Baseline (DeepSeek-V3-03-24). **Recovery Rate** denotes the percentage of successful solutions that initially failed on the first turn but were corrected by the agent loop.

Competition Subset	Baseline (0-Shot)	PCAF Accuracy	Recovery Rate	Avg. Turns
AIME 2025	50.0%	56.67%	29.5%	2.16
BRUMO 2025	N/A	65.83%	48.2%	1.91
CMIMC 2025	N/A	51.25%	41.6%	2.56
HMMT Feb 2025	29.0%	45.83%	35.0%	2.47
HMMT Nov 2025	N/A	46.67%	25.4%	2.28
SMT 2025	N/A	55.66%	36.7%	2.03
Global Average	**39.5%***	**53.65%**	**36.1%**	**2.24**

*Average calculated based on available benchmark data (AIME and HMMT Feb).

The data highlights a significant "Verification Cost": on average, the system required 2.24 turns to reach a final answer. This latency confirms that the default Zero Shot reasoning (Turn 1) is frequently insufficient for these high complexity problems, necessitating the intervention of the Verifier and Planner.

5.2 Resilience Analysis

A critical contribution of PCAF is its resilience. We define the *Recovery Rate* as the proportion of successful runs where the system initially failed (Turn 1) but successfully self corrected in subsequent turns.

- **One-Shot Success (Turn 1):** 62.6% of correct answers were achieved immediately, typically on problems where the analytical derivation was flawless.
- **Collaborative Recovery (Turns 2-4): 36.1%** of correct answers were "rescued" by the agent loop. Without the PCAF architecture, these instances would have been failures.

This 36.1% recovery rate empirically validates the utility of the Planner's deterministic routing logic.

5.3 Case Study: The "Double Check" Effect

We highlight a representative recovery trace from the HMMT dataset (Problem ID 1: Sum of divisors of 9! ending in 1) to illustrate the framework's mechanics.

- **Initial Failure (Solver):** The Solver analytically derived divisors $\{1, 81\}$ (Sum: 82), missing the combination $3 \times 7 = 21$.
- **Conflict Detection (Verifier):** The Verifier's dual method check flagged a `METHOD_CONFLICT`: "Text derivation (82) does not match Code Output (103)."
- **Correction (Planner):** The Planner issued a mandatory instruction: "*CRITICAL CONFLICT... Ignore previous text... Re-derive using code evidence.*"
- **Resolution (Turn 2):** The Solver synthesized the code output $\{1, 21, 81\}$, corrected its prime factorization logic, and returned the correct answer of 103.

6 Discussion

The results validate our hypothesis that separating the roles of generation (Solver) and auditing (Verifier) significantly enhances reliability in mathematical reasoning.

Efficacy of the "Constitution of Verification". The recovery rate of 36.1% indicates that the rigid "Constitution"—specifically the requirement that code output must match natural language reasoning—serves as an effective guardrail. In the HMMT case study, a standard CoT model would have hallucinated the answer 82 without a mechanism to flag the missing factor. The `PersistentSolverSandbox` acted as an adversarial truth grounding mechanism, forcing the model to confront the reality of its calculation errors.

The Planner's Deterministic Routing. Standard self correction often fails because LLMs are "stubborn," frequently repeating their initial error. The PCAF Planner overcomes this by translating vague Verifier critiques into mandatory, strategy shifting instructions. The high success rate on AIME Number Theory tasks (where the Planner often forced a "Brute Force" check) suggests that deterministic strategy switching is more effective than open ended "try again" prompting.

Limitations in Geometry. While the system excelled in Algebra and Number Theory (56.7% on AIME), qualitative analysis of failures reveals a weakness in Geometry. When the Solver attempted to parameterize complex shapes (e.g., AIME Problem 2) in Python, it often struggled to translate visual constraints into correct coordinate code. The Verifier correctly flagged inconsistencies, but the Planner lacked a visual modality to offer spatial correction, leading to loop exhaustion. This suggests that for geometric reasoning, the text based code sandbox must be augmented with Vision Language Models (VLMs).

7 Conclusion

We introduced the Prompt Based Collaborative Agent Framework as a targeted mechanism to improve the reliability of LLMs on the MathArena benchmark. Our results prove that by moving from a linear baseline to a structured, architecture driven collaborative loop, we can achieve a substantial accuracy uplift. With a global accuracy of 53.65% and the ability to recover over one third of initial failures, PCAF demonstrates that architectural oversight is a high value technique for transforming model performance on complex, competitive mathematical tasks.

References

[1] Vincent Li, Yule Fu, Tim Knappe, Kevin Han, and Kevin Zhu. Automating mathematical proof generation using large language model agents and knowledge graphs, 2025. arXiv preprint.

[2] Sumanth Varambally, Thomas Voice, Yanchao Sun, Zhifeng Chen, Rose Yu, and Ke Ye. Hilbert: Recursively building formal proofs with informal reasoning, 2025. arXiv preprint.

[3] Ruida Wang, Rui Pan, Yuxin Li, Jipeng Zhang, Yizhen Jia, Shizhe Diao, Renjie Pi, Junjie Hu, and Tong Zhang. Ma-lot: Model-collaboration lean-based long chain-of-thought reasoning enhances formal theorem proving, 2025. arXiv preprint.

[4] Yuji Wang et al. Matharena: Evaluating llms on uncontaminated math competitions. *arXiv preprint arXiv:2505.23281*, 2025.